

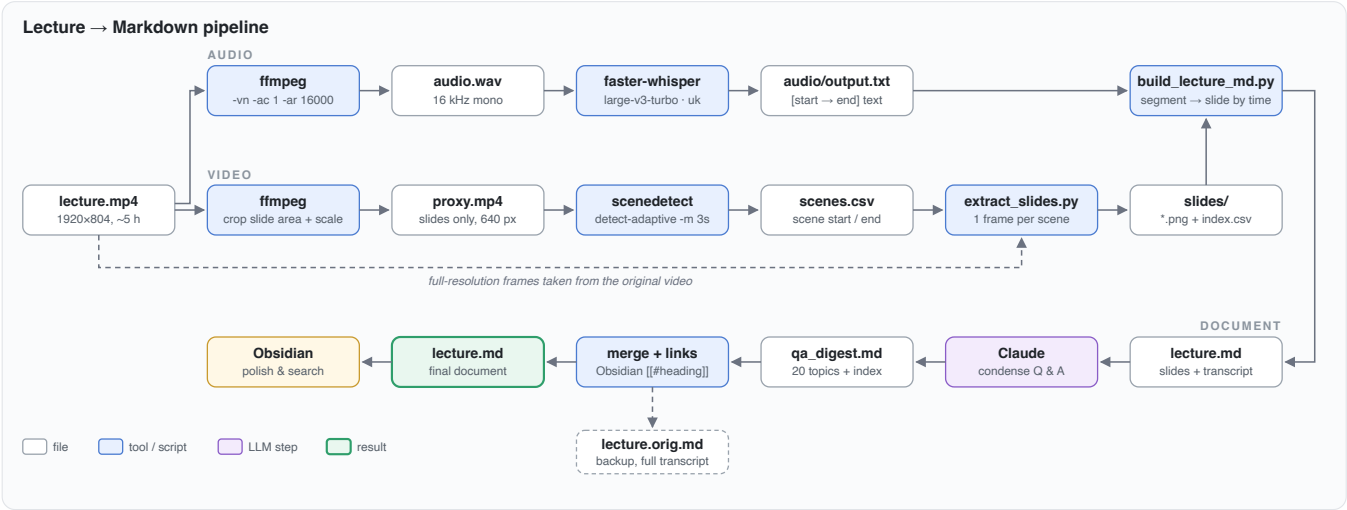
process

Lecture → Markdown: transcription and slide extraction process

How a long recorded lecture (≈5 h, 1920×804, Ukrainian) is turned into a single searchable Markdown document for Obsidian: slide images, the transcript mapped to each slide, and a condensed Q & A digest. Everything uses open-source tools and runs locally.

First run: *Harness Engineering, part 1* (`Harness_Engineering/`).

Pipeline at a glance



Tools

Tool	Purpose
ffmpeg / ffprobe	audio extraction, cropping, proxy video, frame grabs
whisper-ctranslate2 (faster-whisper)	speech-to-text, installed in <code>.venv</code>
scenedetect (PySceneDetect)	slide-change detection, installed in <code>.venv</code>
extract_slides.py	one image per scene from the full-quality video
build_lecture_md.py	maps transcript segments to slides → <code>lecture.md</code>
imagehash, pillow (optional)	near-duplicate slide removal (<code>--dedup</code>)
Claude	Q & A digest, merge, Obsidian link conversion
WhisperModel.faster_whisper	Python tool to transcribe multilingual audio

Folder layout (per lecture)

LLM/	
├ .venv/	whisper-ctranslate2, scenedetect
└ Harness_Engineering/	
├ lecture.mp4	the source video (a symlink to the original works)
├ audio.wav	16 kHz mono audio
├ audio/run.sh	transcription command
├ audio/output.txt	transcript: "[start → end] text" per line
├ proxy.mp4, proxy2.mp4	cropped, downscaled videos for scene detection
├ scenes/run.sh	scene detection command
├ scenes/scenes.csv	scene list
├ slides/	slide images + index.csv
├ extract_slides.py	
├ build_lecture_md.py	
└ qa_digest.md	condensed Q & A (standalone copy)

└─ lecture.orig.md	backup: full transcript version
└─ lecture.md	final document

Steps

1. Measure the video layout

Grab a frame and check where the slides and the speaker overlay sit:

```
ffprobe -v error -show_entries stream=width,height -of csv=p=0 lecture.mp4
ffmpeg -ss 600 -i lecture.mp4 -frames:v 1 probe.png
```

For this recording: frame 1920×804, slides (the desktop share) at **x = 342...1580**, speaker overlay in a corner outside that area.

Check a crop before encoding hours of video:

```
ffmpeg -ss 600 -i lecture.mp4 -vf "crop=1238:804:342:0" -frames:v 1 check.png
```

2. Extract audio

```
ffmpeg -i lecture.mp4 -vn -ac 1 -ar 16000 audio.wav
```

3. Transcribe

3.1. Ukrainian-only attempt - audio/run.sh script

```
whisper-ctranslate2 ../audio.wav --model large-v3-turbo --language Ukrainian \
--output_format all --vad_filter True --condition_on_previous_text False \
--initial_prompt "Лекція про Harness в агентах типу Codex, Claude Code, Pi"
```

The segments are saved to audio/output.txt in the form [0.0 → 22.6] text .

Notes:

- **Set --language explicitly.** Auto-detection only looks at the first 30 s and often confuses Ukrainian and Russian.
- **--initial_prompt** in the lecture's language, listing the English terms, keeps those terms in Latin script instead of transliterated Cyrillic.
- **--condition_on_previous_text False** prevents language drift and repetition loops on long files.
- **--vad_filter True** skips silence and reduces hallucinated text.
- **large-v3-turbo** is fast and good for transcription, but can't translate. Use **large-v3** if you need **--task translate** (to English).
- Mixed-language speech (Russian questions in a Ukrainian lecture) is transcribed as spoken. For per-segment language detection, use the faster-whisper Python API with **multilingual=True**.

3.2. WhisperModel.faster_whisper python tool

```
#!/usr/bin/env python3
import argparse
from faster_whisper import WhisperModel

def main(args):
    model = WhisperModel("large-v3", device="cuda", compute_type="float16")
    segments, info = model.transcribe(
        args.input_fn,
        multilingual=True,                # detect the language for each segment
        vad_filter=True,
        condition_on_previous_text=False,
        initial_prompt="Лекція про Agent Engineering, LLM",
    )
    with open(args.output_fn, "w") as fo:
```

```

    for s in segments:
        fo.write(f"[{s.start:8.1f} → {s.end:8.1f}] {s.text}\n")

if __name__ == '__main__':
    parser = argparse.ArgumentParser(prog='transcribe')
    parser.add_argument('-i', dest='input_fn', help='Input file.wav', required=True)
    parser.add_argument('-o', dest='output_fn', help='Output file.txt', required=True)
    args = parser.parse_args()
    main(args)

```

4. Build a proxy video for scene detection (crop.sh)

Crop to the slide area so speaker movement doesn't count as a slide change, and downscale so detection is fast

N.B. CODEC variable contains hardware-accelerated transcoding on MacOS

```

#!/bin/bash -x
if [ $# -eq 0 ] ; then
    echo "Usage: $0 file.mp4"
    exit 1
fi
IN=$1
shift
OUT="{IN%.mp4}_proxy.mp4"

ovw=180
ovh=100
ovx=$(( 1920 - ovw ))
ovy=$(( 804 - ovh ))

CODEC="-c:v h264_videotoolbox -b:v 5M"

ffmpeg -i "$IN" -an \
    -vf "crop=1280:804:320:0,scale=640:-2" \
    $CODEC -crf 30 "$OUT"

```

- `crop=w:h:x:y` — width 1600 – 320 = 1280, full height, offset x = 320.
- `scale=640:-2` keeps the aspect ratio with an even height.
- `h264_videotoolbox` is the macOS hardware encoder and uses `-b:v`; `-crf` applies only to `libx264`.
- Cropping doesn't change the timeline, so proxy timestamps match the original.
- If the overlay overlaps the slide area, black it out after cropping: `drawbox=x=...:y=...:w=...:h=...:color=black:t=fill`.
- If the end of the lecture is a Q & A without slides, the proxy can stop where the slides end (`-t <seconds>`). Here, `proxy2.mp4` covers 0:00–3:52:37.

Agentic_Engineering/crop.sh is the reusable version: `./crop.sh file.mp4 → file_proxy.mp4`.

4.1. ffmpeg running on Intel CPU

The encoder is `-c:v h264_vaapi`. VA-API encodes from frames in GPU memory, so the command also needs a device and a filter chain that ends with `format=nv12,hwupload`.

Your proxy command, with VA-API encoding:

```

ffmpeg -vaapi_device /dev/dri/renderD128 -i lecture.mp4 -an \
    -vf "crop=1238:804:342:0,scale=640:-2,format=nv12,hwupload" \
    -c:v h264_vaapi -rc_mode CQP -qp 28 proxy.mp4

```

- `-vaapi_device /dev/dri/renderD128` is the render node of the GPU. Run `ls /dev/dri` to see which ones you have.
- `crop` and `scale` run on the CPU, which is cheap at this size. `format=nv12,hwupload` then hands the frames to the GPU.
- `-rc_mode CQP -qp 28` is the VA-API equivalent of `-crf` (lower number = better quality, bigger file); `h264_vaapi` doesn't support `-crf`. For a target bitrate instead, use `-rc_mode VBR -b:v 2M -maxrate 3M`.

Decoding on the GPU too. You can also let the GPU decode the original video, then crop and scale on the CPU as before:

```
ffmpeg -hwaccel vaapi -vaapi_device /dev/dri/renderD128 -i lecture.mp4 -an \
-vf "crop=1238:804:342:0,scale=640:-2,format=nv12,hwupload" \
-c:v h264_vaapi -rc_mode CQP -qp 28 proxy.mp4
```

The decoded frames come back to system memory here, so `crop` and `scale` work unchanged.

Checking what the GPU supports. Run `vainfo`, from the `vainfo` package; Intel GPUs also need the `intel-media-va-driver-non-free` driver on Ubuntu. Look for H.264 entries:

- `VAProfileH264Main : VAEntrypointEncSlice` means normal encoding works.
- If you only see `VAEntrypointEncSliceLP`, the GPU supports only the low-power encoder. This is common on newer Intel iGPUs, including the N100/N150 class. Add `-low_power 1` or the encode will fail.

Other VA-API encoders work the same way: `hevc_vaapi`, `av1_vaapi`, `vp9_vaapi`, depending on the GPU. On NVIDIA hardware, use `NVENC` (`h264_nvenc`) instead.

4.2. ffmpeg running on AMD Ryzen

Are there options for AMD Ryzen 7 9700X CPU?

Yes, although the 9700X is mostly a CPU. It has a small built-in Radeon GPU (2 compute units), and that GPU includes AMD's video engine, **VCN**. On Linux, VCN is used through VA-API (Mesa's `radeonsi` driver), so the same `h264_vaapi` commands from before work.

1. Hardware encoding with the built-in GPU:

```
ffmpeg -hwaccel vaapi -vaapi_device /dev/dri/renderD128 -i lecture.mp4 -an \
-vf "crop=1238:804:342:0,scale=640:-2,format=nv12,hwupload" \
-c:v h264_vaapi -rc_mode CQP -qp 28 proxy.mp4
```

- **Driver:** install `mesa-va-drivers` and `vainfo` on Ubuntu, then check that `vainfo` lists `VAProfileH264Main : VAEntrypointEncSlice`. HEVC encoding should also be available (`hevc_vaapi`). As far as I know this GPU generation decodes AV1 but can't encode it; `vainfo` will tell you.
- **BIOS:** the integrated graphics must be enabled there. On some boards it's switched off automatically when a separate graphics card is installed.
- **With a separate AMD/NVIDIA card:** the built-in GPU may be `renderD129` instead of `renderD128`. Run `vainfo --display drm --device /dev/dri/renderD129` to check which is which.
- `-low_power` is an Intel-only option; AMD doesn't need it.
- `h264_amf` (AMD's own encoder) needs AMD's proprietary runtime on Linux. VA-API is the simpler, standard route.

2. Plain CPU encoding, probably just as good here. An 8-core Zen 5 encodes a 640-px proxy with `libx264` much faster than real time:

```
ffmpeg -i lecture.mp4 -an \
-vf "crop=1238:804:342:0,scale=640:-2" \
-c:v libx264 -preset veryfast -crf 30 proxy.mp4
```

For this job, the slow part is decoding the full-resolution source, not encoding the small proxy. The biggest gain is usually `-hwaccel vaapi` **for decoding**, even if you keep `libx264` for encoding:

```
ffmpeg -hwaccel vaapi -vaapi_device /dev/dri/renderD128 -i lecture.mp4 -an \
-vf "crop=1238:804:342:0,scale=640:-2" \
-c:v libx264 -preset veryfast -crf 30 proxy.mp4
```

This uses the GPU to decode, `libx264` to encode, and `-crf` works as usual. To find which is fastest on your machine, time each variant on a 10-minute piece (add `-t 600`).

5. Detect slide changes (`scenes/run.sh`)

```
scenedetect -i ../proxy.mp4 -m 3s detect-adaptive list-scenes -f scenes.csv
```

- detect-adaptive handles gradual changes and animations better than detect-content.
- -m 3s (minimum scene length) stops fades and quick flicks from counting as separate slides.

Result for this lecture: 157 scenes.

6. Extract slide images

```
python3 extract_slides.py lecture.mp4 scenes/scenes.csv -o slides
```

- Grabs one full-resolution frame per scene from the **original** video, cropped to the slide area (default --crop 1238:804:342:0).
- --pick end (default) takes the frame 1 s before the scene ends, so slides that build up bullet by bullet are captured complete. Other options: middle, start.
- Files are named with the scene start time: 0002_00h07m00s.png. slides/index.csv stores slide, scene, file, start_s, end_s, frame_s.
- Options: --dedup 6 (moves near-duplicates to slides/_dupes/, needs imagehash), --min-len 5, --format jpg, --jobs 8.
- Takes about 16 s for 157 slides.

7. Map the transcript to slides

```
python3 build_lecture_md.py audio/output.txt slides/index.csv -o lecture.md \
  --title "Harness Engineering, частина 1: Харнес зсередини" --timestamps
```

- Each segment goes to the slide on screen at the segment's midpoint. A slide lasts from its start until the next slide starts.
- Segments are grouped into paragraphs at pauses (--pause 2.0) or at a length limit (--max-chars 700). --timestamps adds [hh:mm:ss] before each paragraph.
- Speech after the last slide goes into a **Q & A** section (--tail-title), split into 5-minute sub-sections (--tail-split 300, 0 = no split).
- The document starts with a contents list: slide number, time, and the first words spoken.

8. Condense the Q & A (LLM step)

The Q & A section (~1 h, ~49 k characters) was condensed by Claude into qa_digest.md:

- 20 topics, each with the question in 1–2 lines, the answer as bullet points, and key terms.
- A table at the top: number, time, topic, keywords.
- Written in the lecture's language (Ukrainian) so it can be searched with the same words as the transcript.
- Names that Whisper probably misheard are kept but marked (*за транскриптом*) for checking against the video.

To reproduce with another LLM: extract the segments after the last slide as h:mm:ss text lines and ask for a topic-by-topic digest in this format.

9. Merge into one document

- Back up the full version: lecture.md → lecture.orig.md.
- Replace everything from the Q & A heading to the end of lecture.md with the digest (title dropped, headings demoted one level).

10. Make links Obsidian-compatible

Obsidian doesn't follow HTML anchors (+ [text](#slide-2)), so all internal links were converted to heading links:

- [[#Slide 2|Slide 2 · 00:07:00]]
- inside tables, \| is used instead of | so the cell doesn't split: [[#1. Агенти для ...\|Агенти для ...]]

Heading rules that keep links reliable:

- No colons in headings: time ranges moved to an italic line under the heading (`*00:07:00-00:10:51*`), and `Oracle:` became `Oracle —`.
- No ```, `*`, `#`, `|`, `^`, `[`, `]` in headings.
- Headings must be unique.

Lessons learned

- Crop to the slide area **before** scene detection. It removes most false slide changes caused by the speaker overlay.
- Use a small proxy for detection and the original for images: fast detection, full-quality slides.
- Check that the proxy's duration matches what you expect (`ffprobe -show_entries format=duration`). A shorter proxy silently leaves the rest of the lecture without slides.
- Set the Whisper language and an initial prompt with domain terms. Turn off `condition_on_previous_text` for long recordings.
- The `[start → end]` transcript plus `index.csv` timestamps are all that's needed to align speech with slides.

To do

- Add an `--obsidian` option to `build_lecture_md.py` so it writes heading links directly (it still writes HTML anchors, and step 10 was a one-off conversion).
- Save the merge and link-conversion steps (9–10) as a script.
- Apply the same process to `Agentic_Engineering` (part 1 — theory, part 2 — practice).